

A Brief Introduction to Dynamic Programming



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Santiago Marco-Sola

2. Contents, Objectives, and Main Concepts

Contents

1. Motivational Problem: Computing Fibonacci Numbers
 2. Elements of Dynamic Programming
 3. Five steps to Dynamic Programming
 4. Summary, Next topics, and References.
-

Objectives

1. Understand Dynamic Programming technique to compute recursive problems.
2. Identify the Elements of Dynamic Programming.
3. Learn how to tackle a problem using Dynamic Programming.
4. Practice with example.

Main Concepts

- Dynamic programming
- Memoization
- Subproblem decomposition
- Optimal sub-structure
- Overlapped subproblems

3. Fibonacci Numbers. A Motivational Problem

- The Fibonacci sequence pattern can be observed in many natural events and constructs.
- In mathematics, Fibonacci numbers appear unexpectedly often in many problems and areas.
- Classical Recursive formulation.

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2} \quad (n \geq 2)$$

- That is:

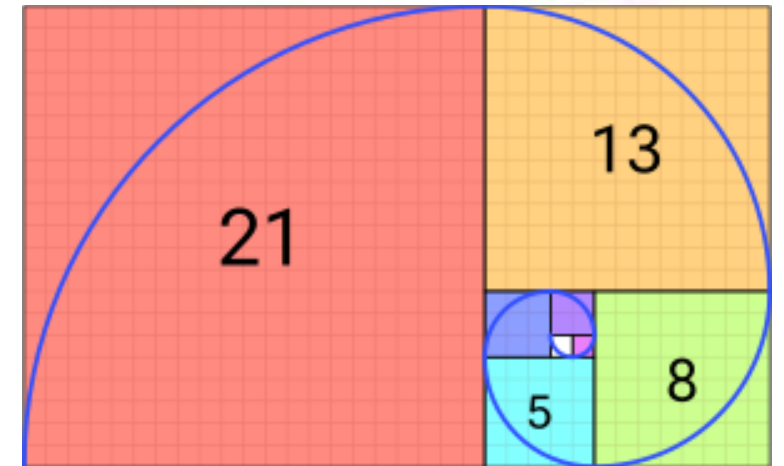
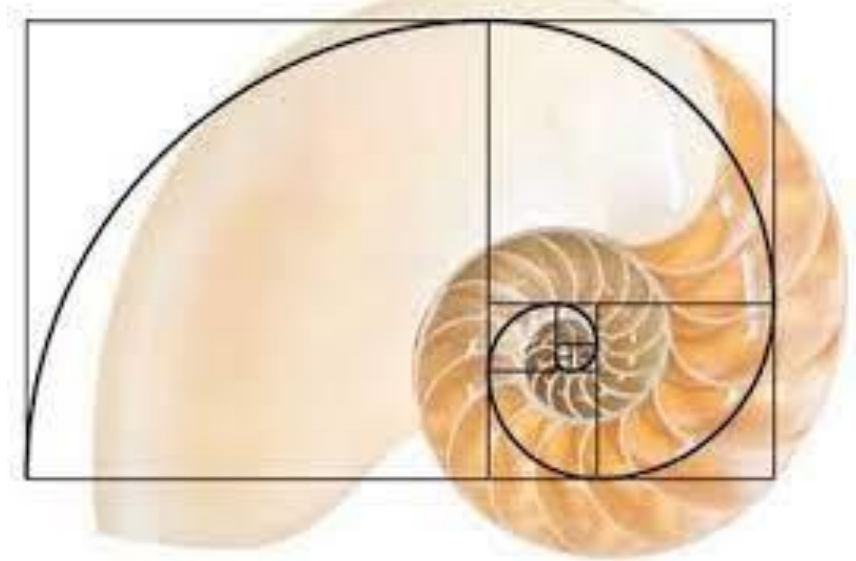
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, ...

Recommended Video (for fun!)

- **[Video]** Nature by Numbers | The Golden Ratio and Fibonacci Numbers

<https://www.youtube.com/watch?v=me6Dnl2DOtM>

$$\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \varphi = 1.661803$$



4. Fibonacci Numbers. A recursive (brute-force) solution

- **Problem:** How to compute the n-th Fibonacci number.
- **Straightforward answer:** Recursive algorithm from formula.

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

- **Result:** $F(n) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$

4. Fibonacci Numbers. A recursive (brute-force) solution

- **Problem:** How to compute the n-th Fibonacci number.
- **Straightforward answer:** Recursive algorithm from formula.

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

- **Result:** $F(n) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$
- **Complexity:** $T(n) = T(n-1) + T(n-2) + 1$
- **Closed-form:** $T(n) \rightarrow 1, 1, 3, 5, 9, 15, 25, 41, 67, \dots$
 $T(n) = 2F(n+1) - 1$ (*verify by induction!*)
- As $F(n+1)/F(n) \sim \varphi \sim 1.61803$
 $F(n) = \theta(\varphi^n)$ (*proving this is not simple!*)
 $O(T(n)) = O(F(n)) = O(1.6^n)$

$O(1.6^n)$ Time
 $O(n)$ Space

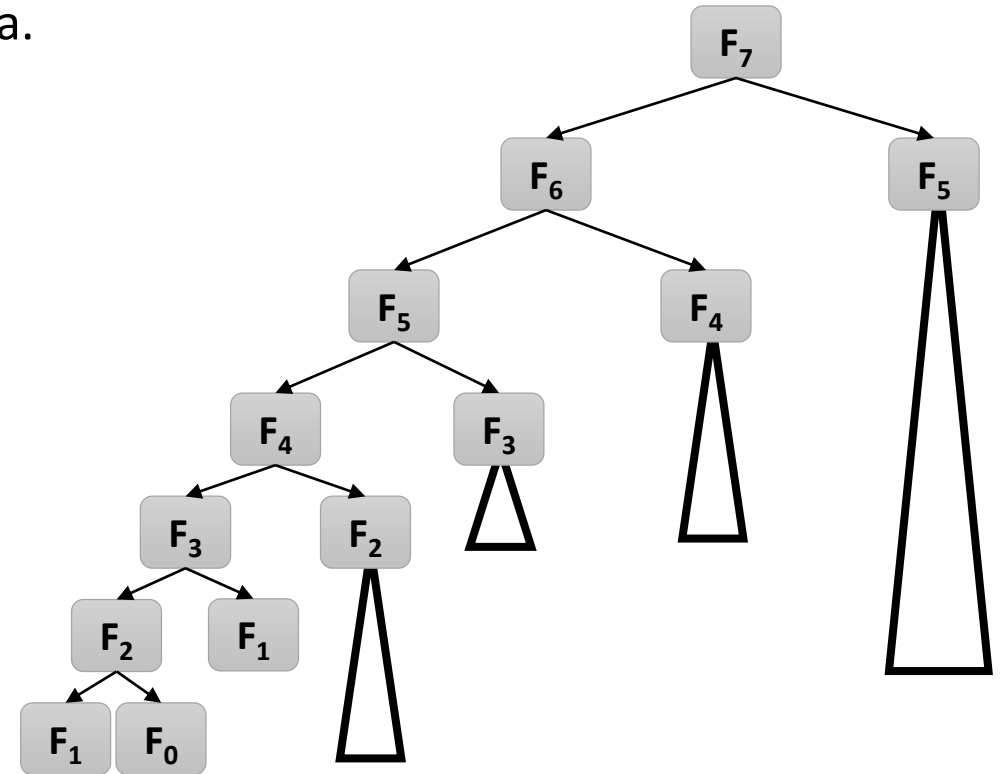
4. Fibonacci Numbers. A recursive (brute-force) solution

- **Problem:** How to compute the n-th Fibonacci number.
- **Straightforward answer:** Recursive algorithm from formula.

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

- **Result:** $F(n) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$
- **Complexity:** $T(n) = T(n-1) + T(n-2) + 1$
- **Closed-form:** $T(n) \rightarrow 1, 1, 3, 5, 9, 15, 25, 41, 67, \dots$
 $T(n) = 2F(n+1) - 1$ (*verify by induction!*)
- As $F(n+1)/F(n) \sim \varphi \sim 1.61803$
 $F(n) = \theta(\varphi^n)$ (*proving this is not simple!*)
 $O(T(n)) = O(F(n)) = O(1.6^n)$



$O(1.6^n)$ Time
 $O(n)$ Space

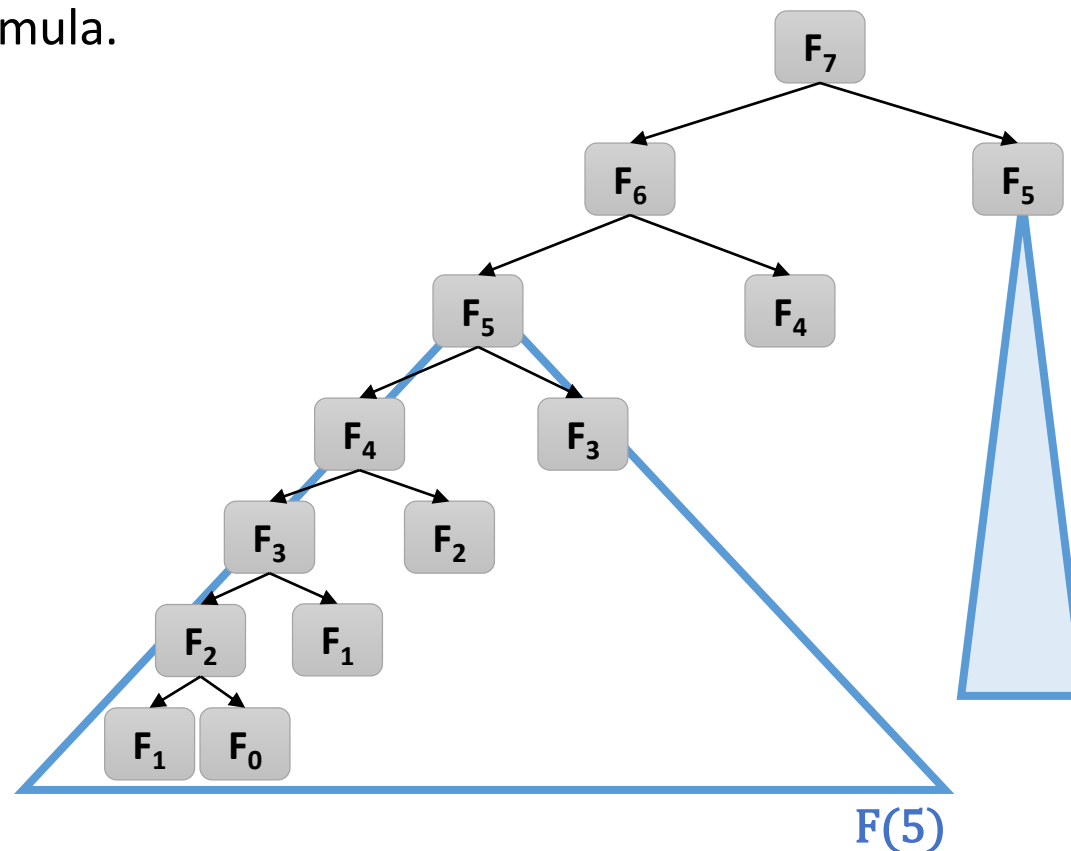
4. Fibonacci Numbers. Repeated Computations!!

- **Problem:** How to compute the n-th Fibonacci number.
- **Straightforward answer:** Recursive algorithm from formula.

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

- **Result:** $F(n) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$
- **Complexity:** $T(n) = T(n-1) + T(n-2) + 1$
- **Closed-form:** $T(n) \rightarrow 1, 1, 3, 5, 9, 15, 25, 41, 67, \dots$
 $T(n) = 2F(n+1) - 1$ (*verify by induction!*)
- As $F(n+1)/F(n) \sim \varphi \sim 1.61803$
 $F(n) = \theta(\varphi^n)$ (*proving this is not simple!*)
 $O(T(n)) = O(F(n)) = O(1.6^n)$



$O(1.6^n)$ Time
 $O(n)$ Space

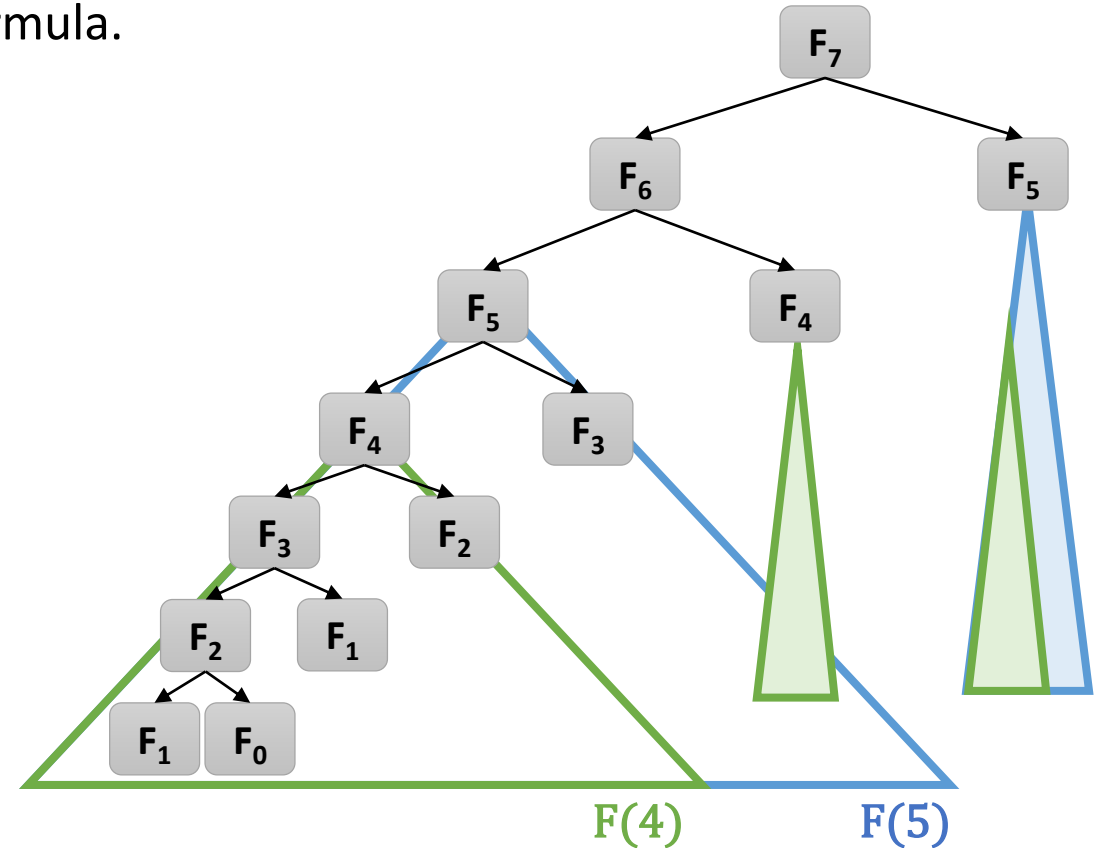
4. Fibonacci Numbers. Repeated Computations!!

- **Problem:** How to compute the n-th Fibonacci number.
- **Straightforward answer:** Recursive algorithm from formula.

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

- **Result:** $F(n) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$
- **Complexity:** $T(n) = T(n-1) + T(n-2) + 1$
- **Closed-form:** $T(n) \rightarrow 1, 1, 3, 5, 9, 15, 25, 41, 67, \dots$
 $T(n) = 2F(n+1) - 1$ (*verify by induction!*)
- As $F(n+1)/F(n) \sim \varphi \sim 1.61803$
 $F(n) = \theta(\varphi^n)$ (*proving this is not simple!*)
 $O(T(n)) = O(F(n)) = O(1.6^n)$



$O(1.6^n)$ Time
 $O(n)$ Space

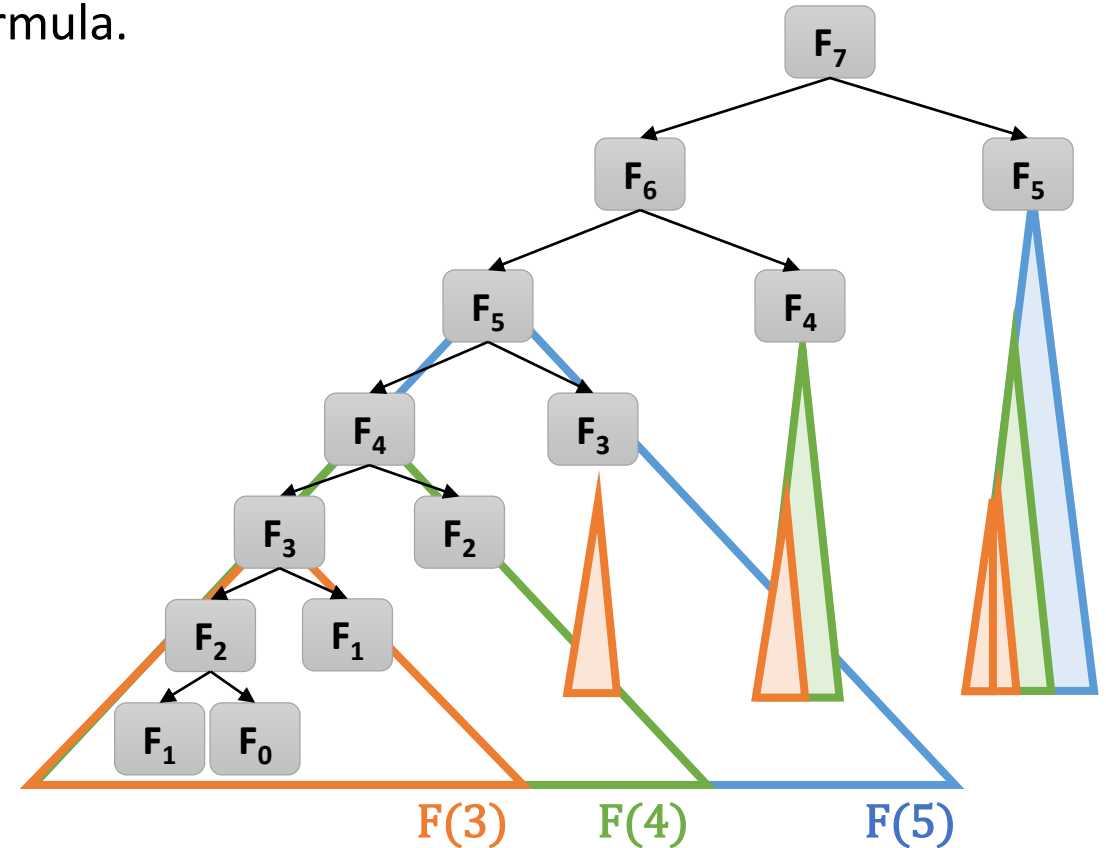
4. Fibonacci Numbers. Repeated Computations!!

- **Problem:** How to compute the n-th Fibonacci number.
- **Straightforward answer:** Recursive algorithm from formula.

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```

- **Result:** $F(n) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$
- **Complexity:** $T(n) = T(n-1) + T(n-2) + 1$
- **Closed-form:** $T(n) \rightarrow 1, 1, 3, 5, 9, 15, 25, 41, 67, \dots$
 $T(n) = 2F(n+1) - 1$ (*verify by induction!*)
- As $F(n+1)/F(n) \sim \varphi \sim 1.61803$
 $F(n) = \theta(\varphi^n)$ (*proving this is not simple!*)
 $O(T(n)) = O(F(n)) = O(1.6^n)$



$O(1.6^n)$ Time
 $O(n)$ Space

5. Fibonacci Numbers. Memoization

- **Strategy:** Memorize already computed values in an array or list. By design, the top-down recurrence for $F(n)$ is evaluated only once for each index.

Fibonacci (Memoization)

```
def fib_memo(F,n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        # Is F[n] already computed?  
        if F[n] is None:  
            F[n] = fib_memo(F,n-1) + fib_memo(F,n-2)  
        # Memorized F[n]  
        return F[n]  
  
def fib_memoization(n):  
    F = [None for _ in range(0,n+1)]  
    return fib_memo(F,n)
```

- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.



$O(n)$ Time
 $O(n)$ Space

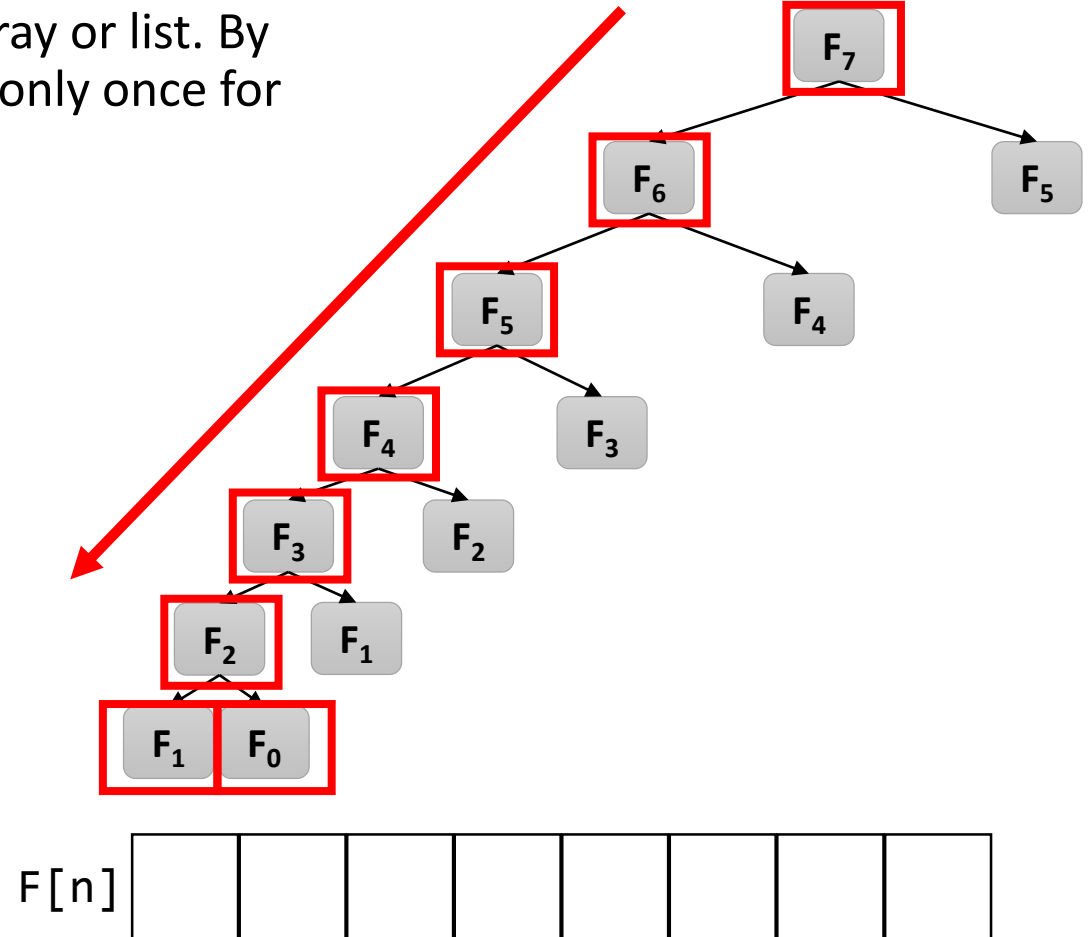
5. Fibonacci Numbers. Memoization

- **Strategy:** Memorize already computed values in an array or list. By design, the top-down recurrence for $F(n)$ is evaluated only once for each index.

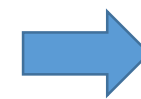
Fibonacci (Memoization)

```
def fib_memo(F,n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        # Is F[n] already computed?  
        if F[n] is None:  
            F[n] = fib_memo(F,n-1) + fib_memo(F,n-2)  
        # Memorized F[n]  
        return F[n]  
  
def fib_memoization(n):  
    F = [None for _ in range(0,n+1)]  
    return fib_memo(F,n)
```

Top-down Memoization



- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.

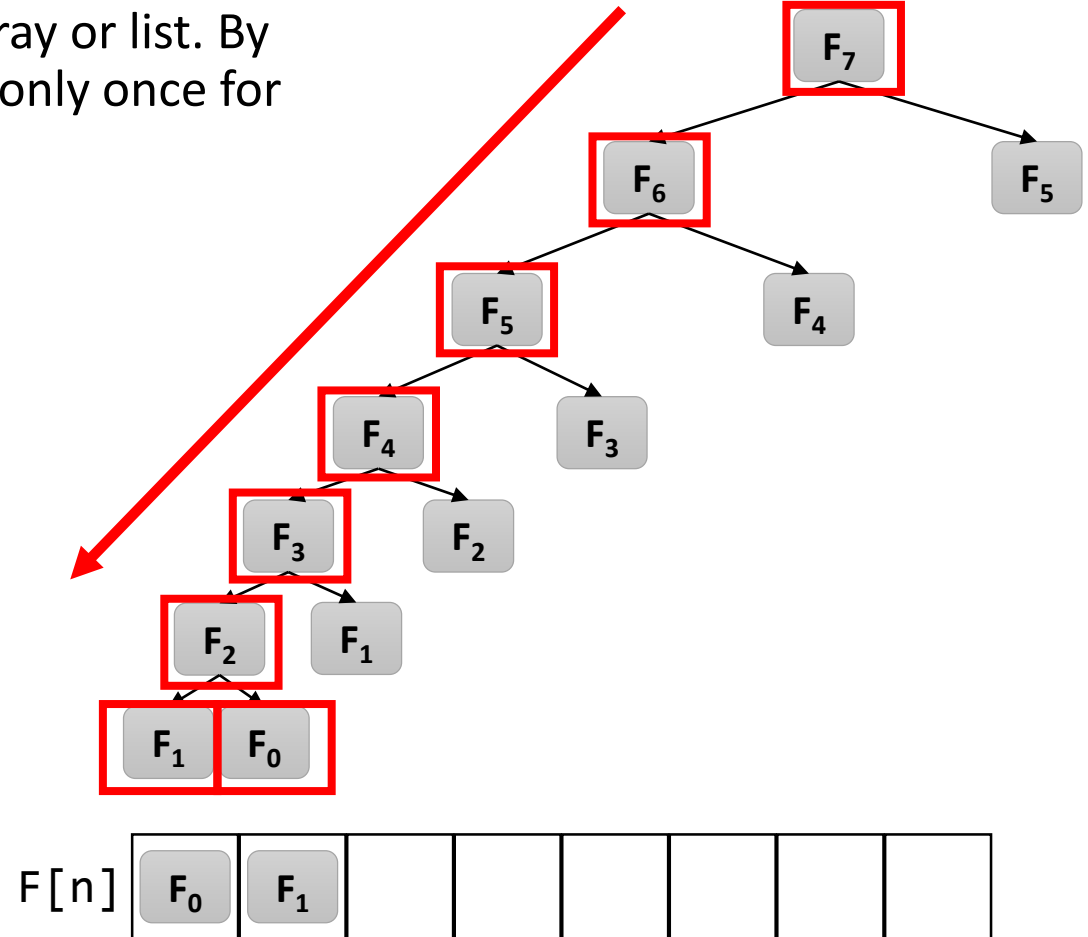


$O(n)$ Time
 $O(n)$ Space

5. Fibonacci Numbers. Memoization

- **Strategy:** Memorize already computed values in an array or list. By design, the top-down recurrence for $F(n)$ is evaluated only once for each index.

Fibonacci (Memoization)	Top-down Memoization
<pre>def fib_memo(F,n): if n == 0: return 0 elif n == 1: return 1 else: # Is F[n] already computed? if F[n] is None: F[n] = fib_memo(F,n-1) + fib_memo(F,n-2) # Memorized F[n] return F[n] def fib_memoization(n): F = [None for _ in range(0,n+1)] return fib_memo(F,n)</pre>	



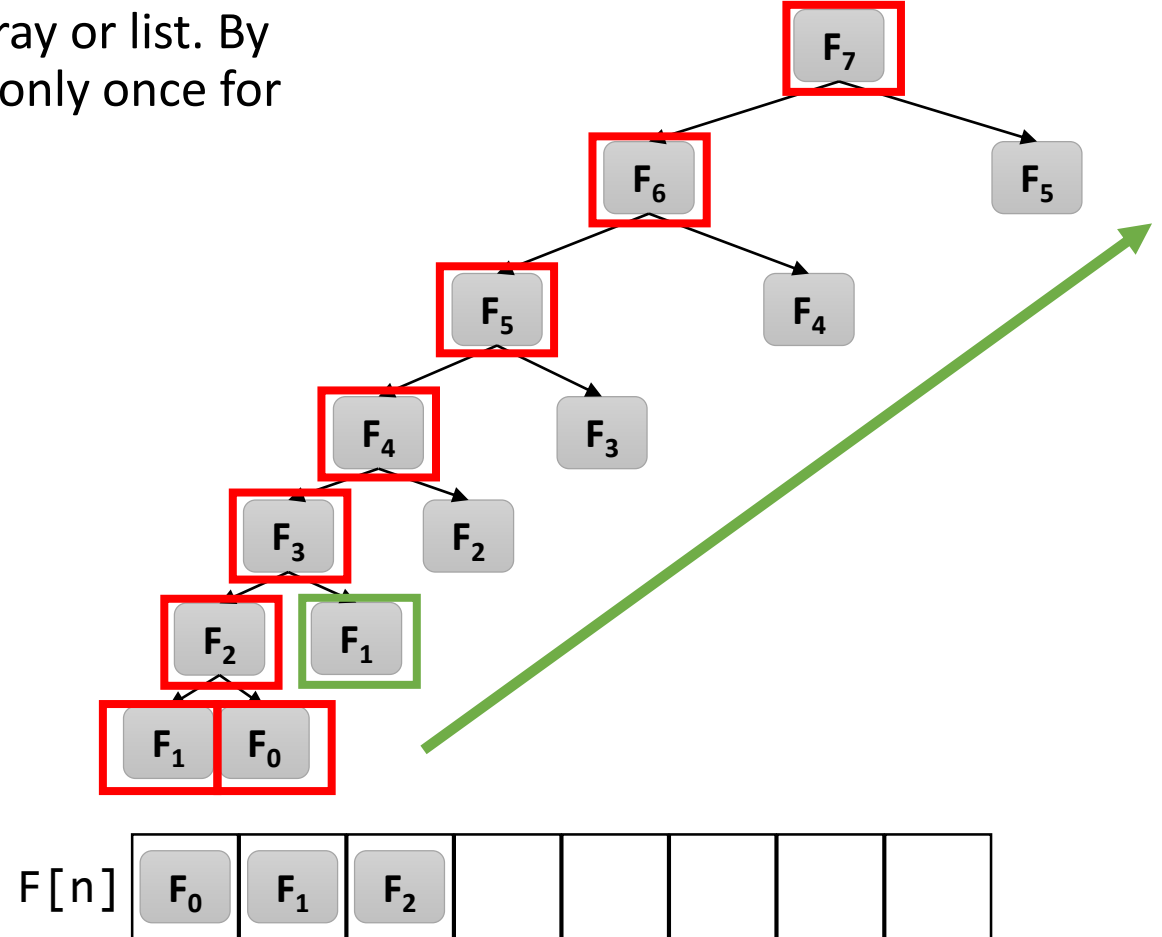
- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.

➡ $O(n)$ Time
 $O(n)$ Space

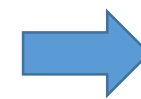
5. Fibonacci Numbers. Memoization

- **Strategy:** Memorize already computed values in an array or list. By design, the top-down recurrence for $F(n)$ is evaluated only once for each index.

Fibonacci (Memoization)	Top-down Memoization
<pre>def fib_memo(F,n): if n == 0: return 0 elif n == 1: return 1 else: # Is F[n] already computed? if F[n] is None: F[n] = fib_memo(F,n-1) + fib_memo(F,n-2) # Memorized F[n] return F[n] def fib_memoization(n): F = [None for _ in range(0,n+1)] return fib_memo(F,n)</pre>	



- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.

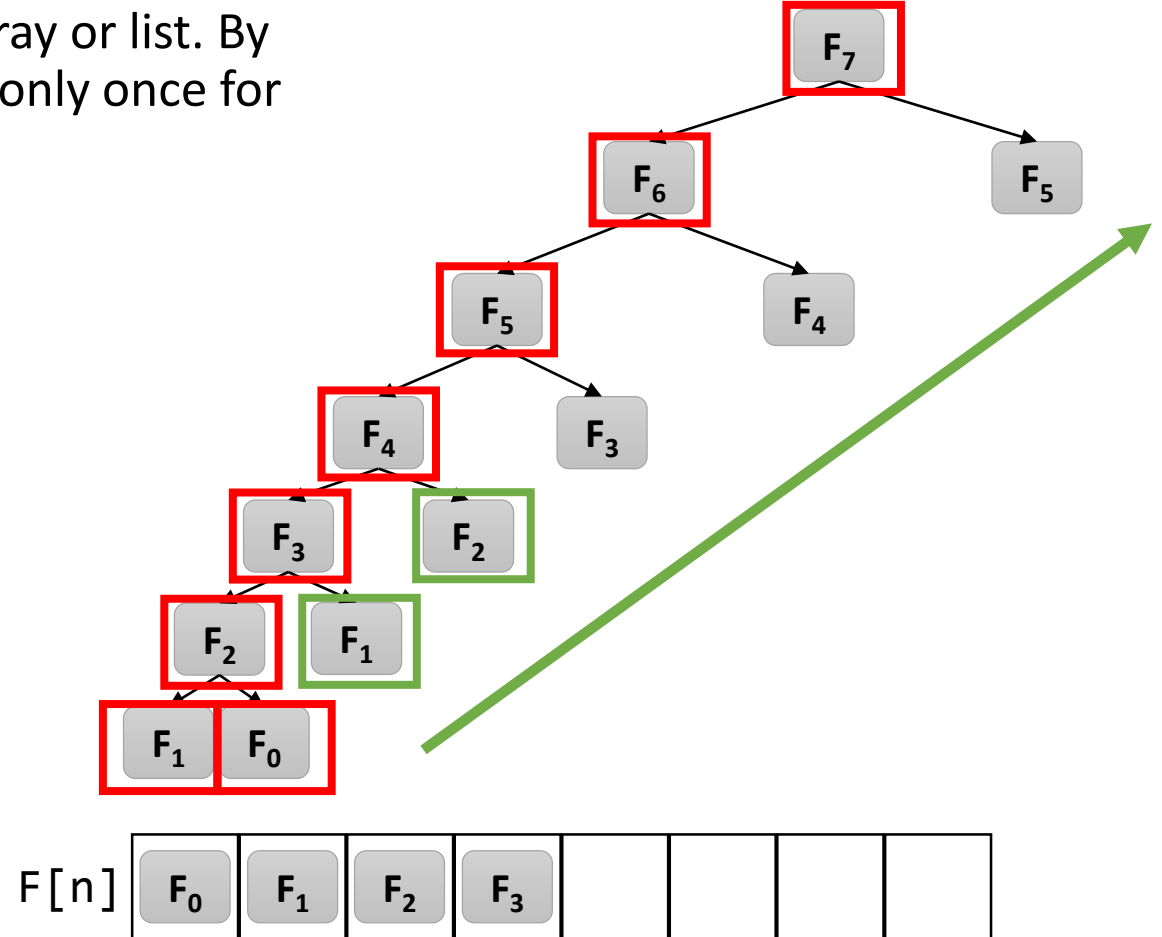


$O(n)$ Time
 $O(n)$ Space

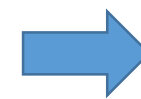
5. Fibonacci Numbers. Memoization

- **Strategy:** Memorize already computed values in an array or list. By design, the top-down recurrence for $F(n)$ is evaluated only once for each index.

Fibonacci (Memoization)	Top-down Memoization
<pre>def fib_memo(F,n): if n == 0: return 0 elif n == 1: return 1 else: # Is F[n] already computed? if F[n] is None: F[n] = fib_memo(F,n-1) + fib_memo(F,n-2) # Memorized F[n] return F[n] def fib_memoization(n): F = [None for _ in range(0,n+1)] return fib_memo(F,n)</pre>	



- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.

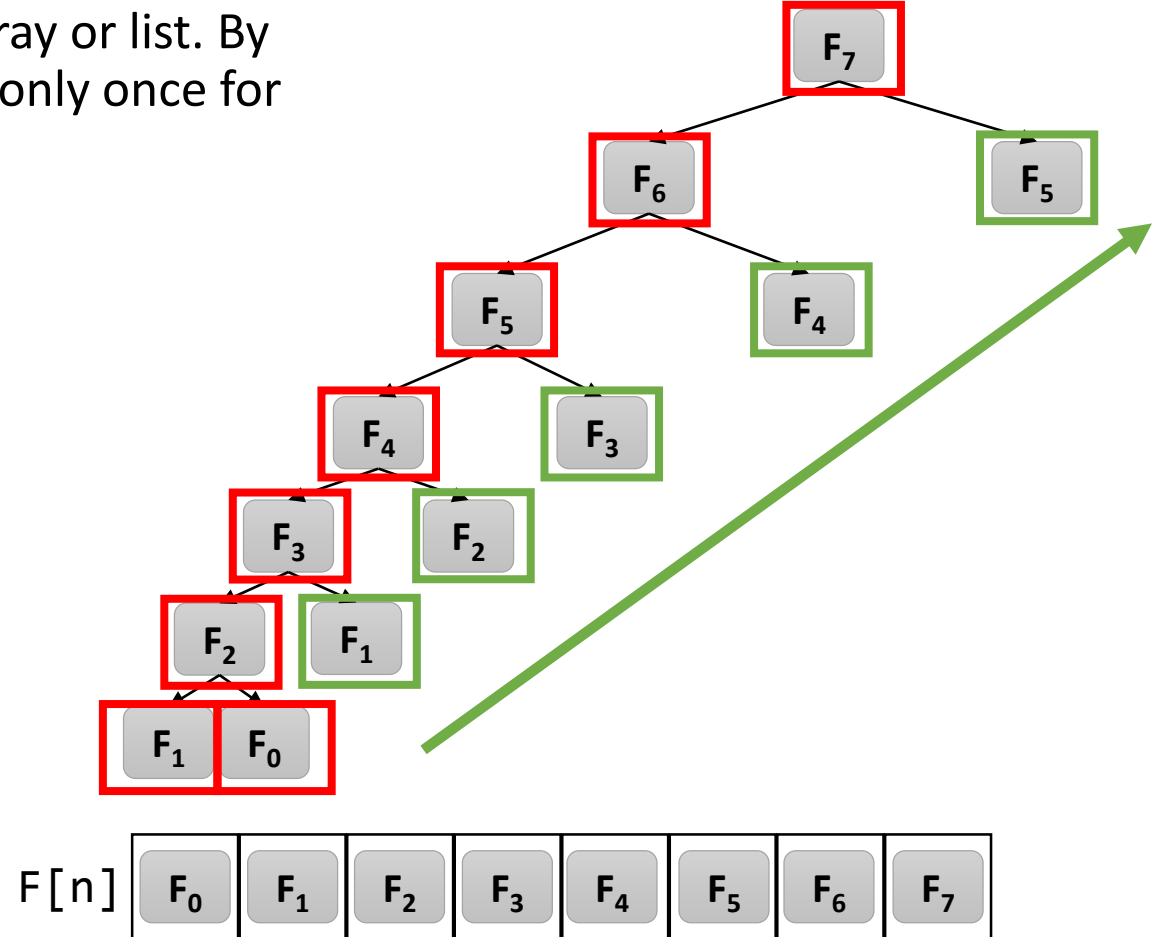


$O(n)$ Time
 $O(n)$ Space

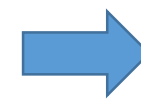
5. Fibonacci Numbers. Memoization

- **Strategy:** Memorize already computed values in an array or list. By design, the top-down recurrence for $F(n)$ is evaluated only once for each index.

Fibonacci (Memoization)	Top-down Memoization
<pre>def fib_memo(F,n): if n == 0: return 0 elif n == 1: return 1 else: # Is F[n] already computed? if F[n] is None: F[n] = fib_memo(F,n-1) + fib_memo(F,n-2) # Memorized F[n] return F[n] def fib_memoization(n): F = [None for _ in range(0,n+1)] return fib_memo(F,n)</pre>	



- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.

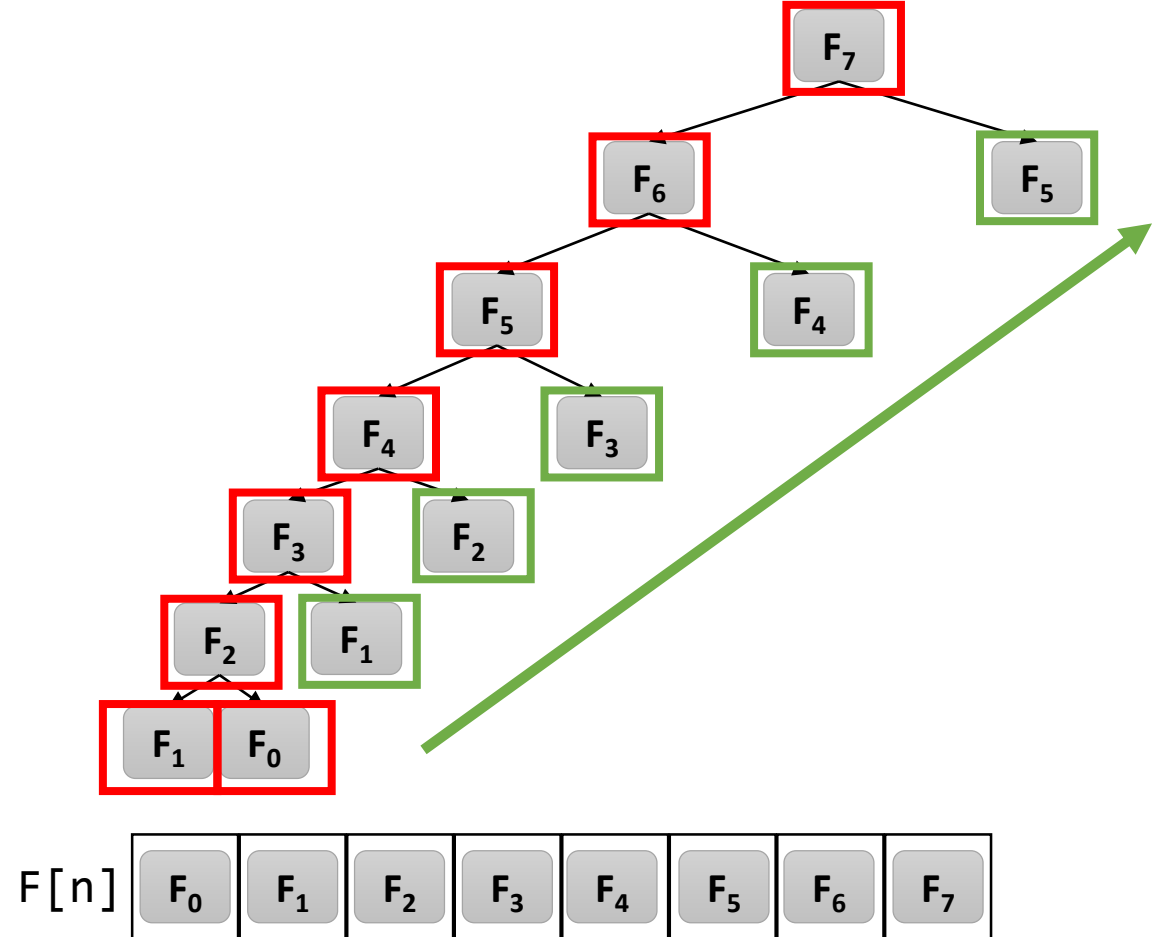


$O(n)$ Time
 $O(n)$ Space

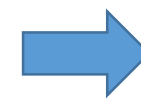
5. Fibonacci Numbers. Memoization

Question

- Do we really need an array to store the values of subproblems (partial solutions)?
- Can we do better than $O(n)$ space?
- How?



- **Complexity:** Each subproblem requires $O(1)$ operations, we have $n + 1$ subproblems, the time cost is $O(n)$.



$O(n)$ Time
 $O(n)$ Space

6. Fibonacci Numbers. “Smart Recursion Computation”

- We observe that the array $F[]$ is filled in increasing order.
- We could just compute $F()$ values in increasing order (bottom-up).

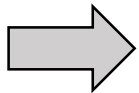
Fibonacci (Dynamic Programming)

```
def fib_dp(n):  
    previous = 0 # F(0)  
    current = 1 # F(1)  
    for i in range(1,n):  
        next_value = previous + current # F(i+1)  
        previous = current  
        current = next_value  
    return current
```

- Complexity:

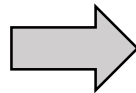
fib_rec(n)

$O(1.6^n)$ Time
 $O(1)$ Space



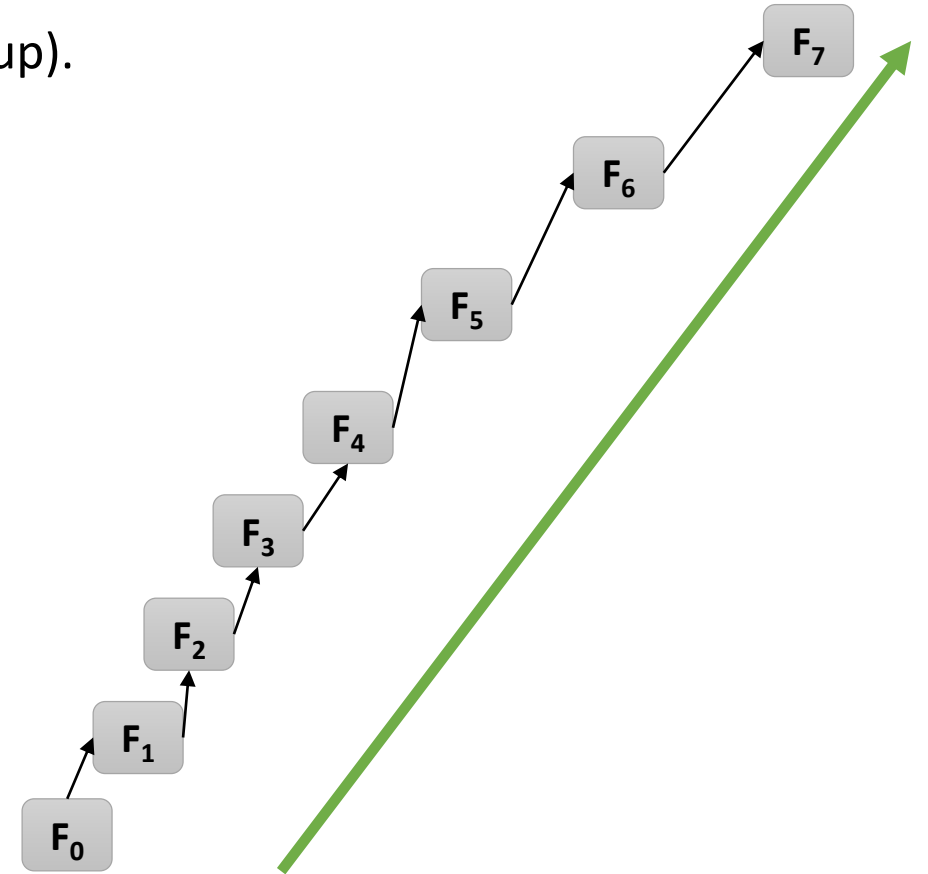
fib_memo(n)

$O(n)$ Time
 $O(n)$ Space



fib_dp(n)

$O(n)$ Time
 $O(1)$ Space



6. Fibonacci Numbers. Dynamic Programming

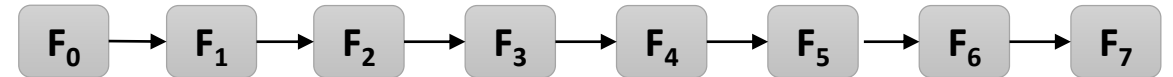
- We observe that the array $F[]$ is filled in increasing order.
- We could just compute $F()$ values in increasing order (bottom-up).

Fibonacci (Dynamic Programming)

```
def fib_dp(n):  
    previous = 0 # F(0)  
    current = 1 # F(1)  
    for i in range(1,n):  
        next_value = previous + current # F(i+1)  
        previous = current  
        current = next_value  
    return current
```

Bottom-up Dynamic Programming

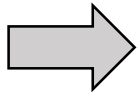
Dynamic programming is not about
filling in arrays/tables.
It's about smart recursion!



• Complexity:

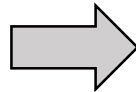
fib_rec(n)

$O(1.6^n)$ Time
 $O(1)$ Space



fib_memo(n)

$O(n)$ Time
 $O(n)$ Space

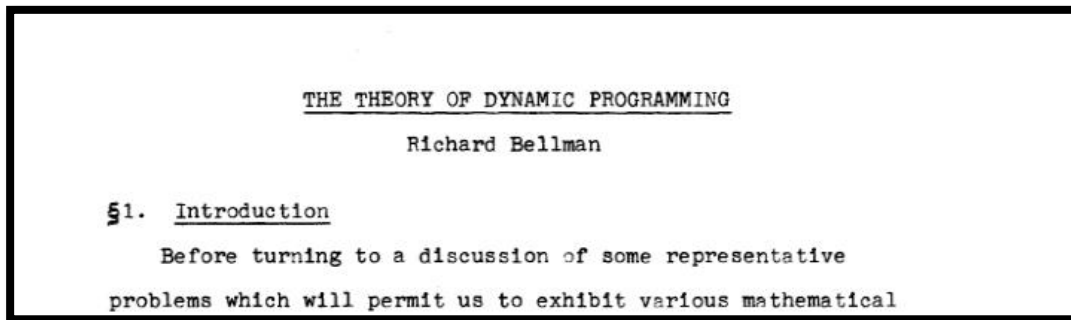


fib_dp(n)

$O(n)$ Time
 $O(1)$ Space

7. Dynamic Programming. Historical Context

- **Dynamic programming** is an algorithmic paradigm or technique developed by Richard Bellman in the 1950s.
 - Applications in numerous fields including economics, aerospace, computational biology, ...
- Dynamic programming is a powerful technique for efficiently compute **recursive problems** storing **partial solutions** to **subproblems** (and **reusing** them)



Bellman, Richard. An introduction to the theory of dynamic programming. RAND CORP SANTA MONICA CA, 1953.

We had a very interesting gentleman in Washington named Wilson. He was secretary of Defense, and he actually had a pathological fear and hatred of the word "research". [...] he would get violent if people used the term "research" or "mathematical" in his presence.

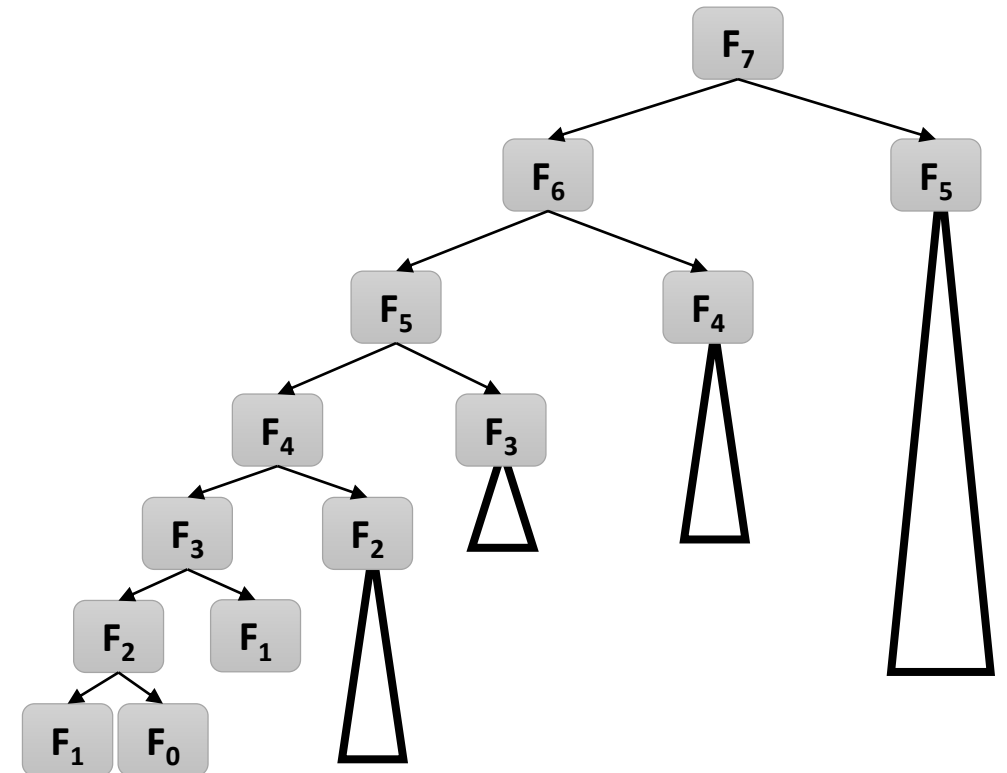
*I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. **What title, what name, could I choose?***

Programming → "planning" or "scheduling"
Dynamic → "multistage" and "sounds cool"



8. Elements of Dynamic Programming

- **Subproblem decomposition.** DP problems need to be divided into subproblems with the same structure.
$$F(n) = F(n-1) + F(n-2)$$
- **Optimal sub-structure.** An optimal solution to the global problem must be a composition of the optimal solution to the subproblems (*Bellman's Principle of Optimality*)
$$\text{Optimal}(F(n-1)) + \text{Optimal}(F(n-2)) \rightarrow \text{Optimal}(F(n))$$
- **Overlapping subproblems.** Recursively dividing the problem into subproblems leads to a “small” number of different problems that **repeated** “a lot”.
E.g., Repeated $F(3)$, $F(3)$, $F(3)$, ...
 - Allows using a memoization data structure to store partial solutions and reuse them (dictionary, array, table, ...)



9. Connections to Other Algorithmic Paradigms

Subproblem decomposition



Optimal sub-structure



Overlapping subproblems

Dynamic Programming (DP)

- Should I forget about **brute-force (exhaustive search) and backtracking**?
 - No, brute-force often is a good starting point to solve the problem. DP can be regarded as an optimization technique for recursive problems.
 - DP does NOT guarantee polynomial cost.
- Is this like **divide & conquer**?
 - D&C recursively divides problems into smaller subproblems.
 - DP doesn't half the size of the problems (often just one step forward in the solution)
 - DP needs repetition of subproblems.
- Is this like **greedy algorithms**?
 - Greedy algorithms take local optimal decisions that lead to global optimal solutions.
 - DP has to explore all subproblems.

10. Five steps to Dynamic Programming

1. Characterize the subproblems

- Same structure (but smaller size).
- Overlapping/Repeated (not an exponential space of subproblems).

2. Formulate the problem recursively.

- Correct recurrence to solve the problem by decomposing into smaller subproblems.

3. Build a top-down solution for the recurrence.

- Choose an appropriated *memorization* data structure.
- Identify dependencies between subproblems.
- Analyze complexity

4. Build a bottom-up solution for the problem.

- Solve subproblems in proper order (and store/reuse them).
- Analyze complexity

5. Backtrack the path to the optimal solution (not covered today)



**You are awesome
Do it right!**

10. Five steps to Dynamic Programming

1. Characterize the subproblems

- Same structure (but smaller size).
- Overlapping/Repeated (not an exponential space of subproblems).

2. Formulate the problem recursively.

- Correct recurrence to solve the problem by decomposing into smaller subproblems.

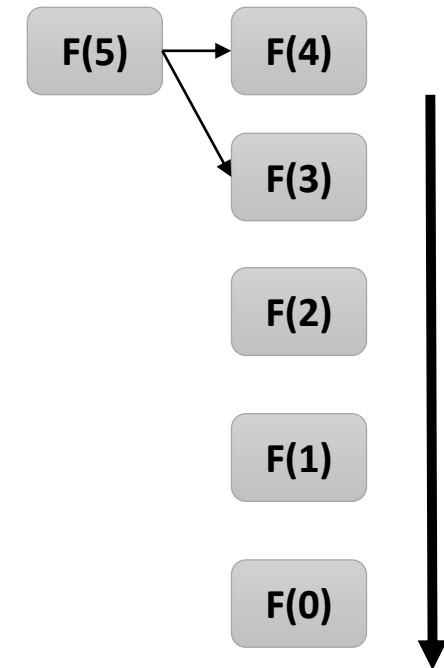
3. Build a top-down solution for the recurrence.

- Choose an appropriated *memorization* data structure.
- Identify dependencies between subproblems.
- Analyze complexity

4. Build a bottom-up solution for the problem.

- Solve subproblems in proper order (and store/reuse them).
- Analyze complexity

5. Backtrack the path to the optimal solution (not covered today)



10. Five steps to Dynamic Programming

1. Characterize the subproblems

- Same structure (but smaller size).
- Overlapping/Repeated (not an exponential space of subproblems).

2. Formulate the problem recursively.

- Correct recurrence to solve the problem by decomposing into smaller subproblems.

3. Build a top-down solution for the recurrence.

- Choose an appropriated *memorization* data structure.
- Identify dependencies between subproblems.
- Analyze complexity

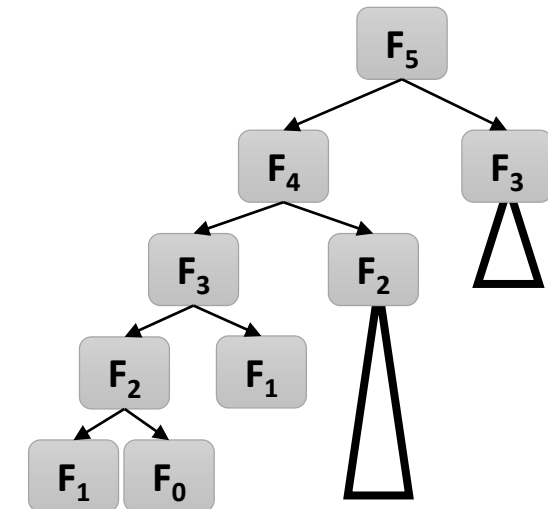
4. Build a bottom-up solution for the problem.

- Solve subproblems in proper order (and store/reuse them).
- Analyze complexity

5. Backtrack the path to the optimal solution (not covered today)

Fibonacci (recursive)

```
def fib_rec(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fib_rec(n-1) + fib_rec(n-2)
```



10. Five steps to Dynamic Programming

1. Characterize the subproblems

- Same structure (but smaller size).
- Overlapping/Repeated (not an exponential space of subproblems).

2. Formulate the problem recursively.

- Correct recurrence to solve the problem by decomposing into smaller subproblems.

3. Build a top-down solution for the recurrence.

- Choose an appropriated *memorization* data structure.
- Identify dependencies between subproblems.
- Analyze complexity

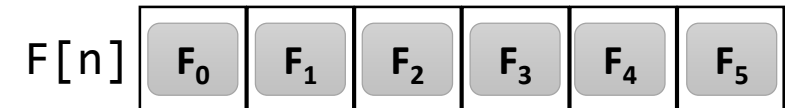
4. Build a bottom-up solution for the problem.

- Solve subproblems in proper order (and store/reuse them).
- Analyze complexity

5. Backtrack the path to the optimal solution (not covered today)

Fibonacci (Memoization)

```
def fib_memo(F,n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        # Is F[n] already computed?  
        if F[n] is None:  
            F[n] = fib_memo(F,n-1) + fib_memo(F,n-2)  
        # Memorized F[n]  
        return F[n]  
  
def fib_memoization(n):  
    F = [None for _ in range(0,n+1)]  
    return fib_memo(F,n)
```



O(n) Time
O(n) Space

10. Five steps to Dynamic Programming

1. Characterize the subproblems

- Same structure (but smaller size).
- Overlapping/Repeated (not an exponential space of subproblems).

2. Formulate the problem recursively.

- Correct recurrence to solve the problem by decomposing into smaller subproblems.

3. Build a top-down solution for the recurrence.

- Choose an appropriated *memorization* data structure.
- Identify dependencies between subproblems.
- Analyze complexity

4. Build a bottom-up solution for the problem.

- Solve subproblems in proper order (and store/reuse them).
- Analyze complexity

5. Backtrack the path to the optimal solution (not covered today)

Fibonacci (Dynamic Programming)

```
def fib_dp(n):  
    previous = 0 # F(0)  
    current = 1 # F(1)  
    for i in range(1,n):  
        next_value = previous + current # F(i+1)  
        previous = current  
        current = next_value  
    return current
```



$O(n)$ Time
 $O(1)$ Space

10. Five steps to Dynamic Programming

1. Characterize the subproblems

- Same structure (but smaller size).
- Overlapping/Repeated (not an exponential space of subproblems).

2. Formulate the problem recursively.

- Correct recurrence to solve the problem by decomposing into smaller subproblems.

3. Build a top-down solution for the recurrence.

- Choose an appropriated *memorization* data structure.
- Identify dependencies between subproblems.
- Analyze complexity

4. Build a bottom-up solution for the problem.

- Solve subproblems in proper order (and store/reuse them).
- Analyze complexity

5. Backtrack the path to the optimal solution (not covered today)

Fibonacci (Dynamic Programming)

```
def fib_dp(n):  
    previous = 0 # F(0)  
    current = 1 # F(1)  
    for i in range(1,n):  
        next_value = previous + current # F(i+1)  
        previous = current  
        current = next_value  
    return current
```



$O(n)$ Time
 $O(1)$ Space

11. Summary

- Dynamic programming is a powerful technique for efficiently compute **recursive problems** storing **partial solutions** to **subproblems** (and **reusing** them).
 - **Subproblems** with the same structure.
 - **Optimal substructure.** Optimal global solution can be constructed from optimal solutions of its subproblems.
 - **Memorize** partial solutions (**repeated subproblems**) and reuse them.
- What is next?
 - Practice the 5 steps to dynamic programming:
 - Longest increasing subsequence
 - Edit-distance
 - Knapsack.
 - **Lab practice:** implementing dynamic programming.

Recommended Materials

[Book]

Introduction to Algorithms. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Chapter 15.

[eBook]

Algorithms. Jeff Erickson.

<http://jeffe.cs.illinois.edu/teaching/algorithms/>

[Video]

Erik Demaine Lecture on Dynamic Programming (MIT courses).

<https://www.youtube.com/watch?v=r4-cftqTcdI>

Questions



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH